

1. ¿Qué es Node.js?

Node.js

- Un framework de frontend
- Un motor para compilar C++
- **Un entorno de ejecución de JavaScript en el servidor**
- Una base de datos NoSQL

Explicación:

Correcta: Un entorno de ejecución de JavaScript en el servidor.

Node combina V8 + APIs de sistema + *event loop* no bloqueante: ideal para E/S intensiva (APIs, websockets, microservicios).

2. ¿Qué motor usa Node.js para ejecutar JavaScript?

Node.js

- SpiderMonkey
- Chakra
- JavaScriptCore
- **V8**

Explicación:

Node integra el motor V8 (Google) para JS.

3. Node.js se caracteriza por...

Node.js

- I/O bloqueante y multihilo por defecto
- **I/O no bloqueante y orientado a eventos**
- Sólo ejecución síncrona
- Necesita JVM para correr

Explicación:

Event loop + APIs asíncronas.

4. ¿Para qué sirve el módulo "fs"?

Node.js

- Rutas de archivos
- **Sistema de archivos (leer/escribir)**
- Servidor HTTP
- CORS

Explicación:

Correcta: Sistema de archivos (leer/escribir).

Ejemplos:

```
fs.readFileSync("data.json", "utf8")
fs.writeFileSync("data.json", JSON.stringify(obj, null, 2))
```

Prefiere versiones asíncronas en producción para no bloquear.

5. ¿Qué hace `JSON.parse(...)` en Node?

Node.js

- Convierte objeto a texto
- **Convierte texto JSON a objeto JS**
- Valida un JSON contra un schema
- Encripta datos

Explicación:

Correcta: Convierte texto JSON a objeto JS.

Complementaria a `JSON.stringify`. Usa `try...catch` para capturar JSON inválido.

6. ¿Qué hace `JSON.stringify(obj, null, 2)`?

Node.js

- Texto → objeto
- **Objeto → texto formateado**
- Minifica JSON
- Valida JSON

Explicación:

Correcta: Objeto → texto formateado.

Convierte un objeto/array a cadena JSON. `null, 2` añade sangría de 2 espacios para legibilidad.

Útil en logs, diffs y persistencia:

```
const json = JSON.stringify({a:1, b:[2,3]}, null, 2);
```

7. ¿Para qué sirve el módulo "path"?

Node.js

- **Gestionar rutas de archivos de forma segura**
- Crear servidores
- Encriptar datos
- Validar JSON

Explicación:

`join/resolve` normalizan rutas.

8. En ES Modules, ¿por qué usamos `fileURLToPath(import.meta.url)`?

Node.js

- **Obtener `__dirname/` `__filename` equivalentes**
- Importar módulos CJS
- Activar TypeScript
- Habilitar CORS

Explicación:

Derivar rutas absolutas sin `__dirname` nativo.

9. ¿Cuál es la forma moderna de importar un módulo?

Node.js

- `import { funcion } from "../modulo.js"`
- `const modulo = require("../modulo")`
- `load("../modulo")`
- `include("../modulo")`

Explicación:

Correcta: `import { funcion } from "../modulo.js"`.

Con ES Modules (ESM), la forma recomendada es `import/export`. Ventajas:

- **Análisis estático:** las herramientas detectan dependencias sin ejecutar.
- **Tree-shaking:** elimina código no usado al empaquetar.
- **Unificación:** mismo sistema en navegador y Node (añadiendo `"type": "module"` en `package.json`).

`require()` de CommonJS sigue existiendo, pero ESM es el estándar moderno.

10. ¿Donde añadimos `"type": "module"`?

Node.js

- `package.json`
- `.env`
- `index.js`
- `nodemon.json`

Explicación:

Define el tipo de módulo.

11. ¿Cómo inicializas un proyecto Node?

Node.js

- `node start`
- `npm init -y`
- `npm run dev`
- `npx serve`

Explicación:

Crea `package.json` por defecto.

12. ¿Qué hace "nodemon"?

Node.js

- Compila TS
- **Reinicia el server al guardar**
- Gestiona DB
- Serve-side rendering

Explicación:

Observa archivos y reinicia proceso.

13. ¿Qué hace fs.readFileSync(ruta, "utf8")?

Node.js

- Lee de forma asíncrona
- **Lee de forma síncrona**
- Escribe asíncrono
- Escribe síncrono

Explicación:

Correcta: Lee de forma síncrona.

Bloquea el hilo hasta acabar. Úsalo puntualmente (scripts, CLI). En servidores, mejor la versión asíncrona.

14. ¿Qué es el event loop?

Node.js

- Planificador del SO
- **Bucle que gestiona la cola de eventos/tareas**
- Mecanismo de clustering
- Una API de streams

Explicación:

Correcta: Bucle que gestiona la cola de eventos/tareas.

Es el corazón del modelo no bloqueante: coordina callbacks, promesas, timers y E/S.

15. ¿Qué es Express.js?

Node.js

- ORM para Node
- **Framework minimalista para servidores HTTP/APIs**
- Librería de React
- CLI de Node

Explicación:

Correcta: Framework minimalista para servidores HTTP/APIs.

Simplifica rutas, manejo de respuestas y uso de middlewares. Base habitual para APIs REST en Node.

16. ¿Cómo se instala Express?

Express

- **npm i express**
- npm start express
- node install express
- npx express

Explicación:

Correcta: npm i express.

Instala la dependencia. Suele acompañarse de `npm i -D nodemon` y scripts en `package.json`.

17. ¿Qué hace `"const app = express()"`?

Express

- Crea un router
- **Crea una instancia de aplicación**
- Crea un servidor WebSocket
- Crea un modelo de datos

Explicación:

Correcta: Crea una instancia de aplicación.

Sobre `app` defines middlewares (`app.use`), rutas (`app.get/post/put/patch/delete`) y escuchas el puerto (`app.listen`).

18. ¿Qué hace `app.listen(3000)`?

Express

- Crea endpoints
- **Inicia el servidor en el puerto 3000**
- Conecta a la DB
- Sirve archivos estáticos

Explicación:

Correcta: Inicia el servidor en el puerto 3000.

Abre un puerto TCP para aceptar peticiones HTTP. Para despliegues usa `process.env.PORT`.

19. ¿Qué es un middleware?

Express

- Base de datos
- **Función que procesa req/res y llama a next()**
- Un template
- Un endpoint

Explicación:

Correcta: Función que procesa req/res y llama a `next()`.

Ejemplos: logger, CORS, autenticación, validación. Se encadenan con `app.use(fn)` o por ruta específica.

20. ¿Para qué sirve `express.json()`?

Express

- **Parsear cuerpo JSON a req.body**
- Habilitar CORS
- Servir estáticos
- Hacer logging

Explicación:

Correcta: Parsear cuerpo JSON a `req.body`.

Sin este middleware, `req.body` llega `undefined` cuando envías JSON. Añádelo:

```
app.use(express.json());
```

21. ¿Qué hace `app.use(cors())`?

Express

- ☐ Bloquea métodos
- ☒ **Habilita peticiones entre orígenes distintos**
- ☐ Habilita HTTPS
- ☐ Define rutas

Explicación:

Permite cross-origin desde el frontend.

22. ¿Qué hace `res.json(data)`?

Express

- ☐ Envía HTML
- ☐ Envía binario
- ☒ **Envía JSON con cabeceras adecuadas**
- ☐ Redirige

Explicación:

Correcta: Envía JSON con cabeceras adecuadas.

Añade `Content-Type: application/json`, serializa el objeto y cierra la respuesta: `res.json({ ok:true })`.

23. Una ruta GET típica en Express:

Express

- ☒ **`app.get("/api", cb)`**
- ☐ `app.fetch("/api", cb)`
- ☐ `app.read("/api", cb)`
- ☐ `app.pull("/api", cb)`

Explicación:

Correcta: `app.get("/api", cb)`.

Los métodos HTTP se mapean a `app.get/post/put/patch/delete`.

24. ¿Qué deposita `app.use(express.static("public"))`?

Express

- ☐ Páginas de DB
- ☒ **Archivos estáticos desde `/public`**
- ☐ Rutas dinámicas
- ☐ Logs a consola

Explicación:

Correcta: Archivos estáticos desde `/public`.

Sirve `public/index.html`, `public/app.js`, `public/styles.css`, etc.

25. ¿Qué hace `res.status(404).send("No encontrado")`?

Express

- ☐ Devuelve 200 OK
- ☒ **Devuelve 404 Not Found con texto**
- ☐ Devuelve 500
- ☐ Redirige 301

Explicación:

Correcta: Devuelve 404 Not Found con texto.

Combina el código de estado y el cuerpo de respuesta en una sola cadena de métodos.

26. CRUD significa:

Express

- ☒ **Create, Read, Update, Delete**
- ☐ Connect, Run, Upload, Download
- ☐ Copy, Remove, Undo, Deploy
- ☐ Compile, Run, Use, Drop

Explicación:

Correcta: Create, Read, Update, Delete.

Mapeo REST típico: `POST /recurso` (Create), `GET /recurso` (Read), `PUT/PATCH /recurso/:id` (Update), `DELETE /recurso/:id` (Delete).

27. Método para crear un recurso:

Express

- ☐ GET
- ☒ **POST**
- ☐ PUT
- ☐ DELETE

Explicación:

POST crea; PUT/PATCH actualizan.

28. Método para leer datos:

Express

- ☐ POST
- ☒ **GET**
- ☐ PATCH
- ☐ DELETE

Explicación:

Correcta: GET.

GET recupera recursos sin efectos colaterales
(idempotente). **POST** crea, **PUT/PATCH** actualizan, **DELETE** elimina.

29. Método para reemplazar totalmente un recurso:

Express

- PATCH
- **PUT**
- POST
- HEAD

Explicación:

Correcta: PUT.

Usa **PUT** para reemplazar el estado completo del recurso (p. ej. PUT /api/users/7 con el objeto completo). **PATCH** aplica cambios parciales.

30. Método para eliminar un recurso:

Express

- **DELETE**
- REMOVE
- DROP
- ERASE

Explicación:

DELETE elimina el recurso.

31. Código 201 significa:

CRUD

- **Creado**
- No autorizado
- No encontrado
- Sin contenido

Explicación:

Correcta: Creado.

Úsalo tras **POST** exitoso; si es posible, envía también la URL del nuevo recurso en Location.

32. ¿Qué hace fs.writeFileSync(...) en Node?

CRUD

- **Escribe un archivo de forma síncrona**
- Escribe un archivo de forma asíncrona
- Lee un archivo
- Elimina un archivo

Explicación:

Correcta: Escribe un archivo de forma síncrona.

En servidores usa la versión asíncrona (`fs.promises.writeFile`) para no bloquear el event loop.

33. ¿Por qué usar CORS?

CRUD

- Proteger contra XSS
- **Permitir o restringir orígenes permitidos**
- Hacer SSR
- Hacer hashing

Explicación:

Correcta: Permitir o restringir orígenes permitidos.

CORS regula qué dominios pueden llamar a tu API desde el navegador. Ajusta cabeceras como: `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`. En Express: `app.use(cors({ origin: "http://tu-frontend" }))`.

No protege de XSS ni cifra; es una política de mismo origen que aplica el navegador.

34. Si no llamas a `express.json()` en POST JSON:

CRUD

- **req.body será undefined**
- Todo funciona igual
- Se activa CORS
- Se borra `req.params`

Explicación:

Correcta: `req.body` será undefined.

El parser es necesario para poblar el cuerpo en peticiones con JSON.

35. ¿Qué devuelve `res.json({ ok:true })`?

CRUD

- Texto plano
- **JSON con content-type application/json**
- HTML
- XML

Explicación:

Correcta: JSON con Content-Type: `application/json`.

Asegura cabeceras apropiadas y serialización estandarizada.

36. ¿Qué hace `try...catch`?

CRUD

- Optimiza CSS
- **Maneja excepciones para evitar caída del proceso**
- Compila TS

- Configura CORS

Explicación:

Correcta: Maneja excepciones para evitar caída del proceso.

Te permite responder con un 500 controlado:

```
try { /* lógica */ }  
catch (e) { res.status(500).json({ error: 'Fallo interno' }); }
```

37. ¿Qué comando lanza nodemon?

CRUD

- node .
- **npx nodemon**
- npm -g node
- npm build

Explicación:

Correcta: npx nodemon.

Puedes crear un script: "dev": "nodemon server.js" y ejecutar npm run dev.

38. Un middleware personalizado es:

CRUD

- Archivo .env
- **Función (req,res,next) con lógica reusable**
- Modelo de datos
- Plantilla HTML

Explicación:

Correcta: Función (req, res, next) reusable.

Ejemplo:

```
function logger(req,res,next){ console.log(req.method, req.url); next(); }  
app.use(logger);
```

39. Buena práctica al responder errores:

CRUD

- No dar detalles y 200 siempre
- **Códigos HTTP correctos y mensaje claro**
- Enviar HTML siempre
- Cerrar el server

Explicación:

Correcta: Códigos HTTP correctos y mensaje claro.

Devuelve el **status** apropiado (400, 404, 500, etc.) y un cuerpo JSON breve y útil.

```
res.status(400).json({ error: "Datos inválidos", detalle: "falta 'email'" });
```

No respondas 200 siempre, no fuerces HTML en APIs y no apagues el server por cada error.

40. Sirviendo frontend desde Express, usas:

CRUD

- `app.use(express.static("dist"))`
- `res.json(dist)`
- `req.body = dist`
- `app.put("dist")`

Explicación:

Correcta: `app.use(express.static("dist"))`.

Publica los archivos ya empaquetados del cliente (Vite/React/etc.).